Fichiers, erreurs et modules

Le programme officiel du Day 2 : organiser son code en fonctions et en fichiers, lire et écrire des fichiers texte, et empêcher un programme de planter quand les données sont mauvaises. C'est exactement ce dont ExamPredict a besoin pour lire les PDF des lycéens sans jamais s'arrêter.

1. Retour sur les fonctions
2. return ou print
3. Lire un fichier
4. Écrire un fichier
5. Les exceptions : try / except
6. Lever sa propre erreur : raise
7. Modules et import
8. Le piège de l'import et __main__
9. Découper un projet en fichiers
10. Mission : l'analyseur de notes
11. À savoir par cœur

---

1. Retour sur les fonctions

Une fonction est un **mixeur** : on le construit une fois avec `def`, puis on l'utilise autant de fois qu'on veut.

> **Définition 1.1** *(paramètre et argument)*
>
> Le **paramètre** est le nom de la case prévue pour l'ingrédient, écrit dans `def`. L'**argument** est l'ingrédient réel qu'on met dedans au moment de l'appel.

```
def saluer(prenom, politesse="Bonjour"):    # prenom et politesse sont des paran
    return f"{politesse} {prenom}"          # politesse a une valeur PAR DÉFAUT

saluer("Lina")                      # 'Bonjour Lina'   ← argument positionnel
saluer("Lina", politesse="Salut")   # 'Salut Lina'     ← argument nommé
```

En clair : une valeur par défaut, c'est le réglage d'usine du mixeur. On peut le changer, mais on n'est pas obligé.

> **Piège (vu en F1.DAY1, section 6)**
>
> Jamais de valeur par défaut **modifiable** comme `liste=[]` : la même liste est réutilisée à chaque appel. On écrit `liste=None`, puis `if liste is None: liste = []`.

2. `return` ou `print`

	`PRINT`	`RETURN`
Image	Montrer le plat au client	Donner le plat au client
Après	la valeur est perdue	on peut la ranger, la réutiliser, la tester
Ce que renvoie la fonction	`None` (rien)	la valeur

```
def moyenne_print(notes):
    print(sum(notes) / len(notes))

m = moyenne_print([10, 20])    # affiche 15.0...
print(m + 1)                   # TypeError : m vaut None !
```

> **Règle**
>
> Une fonction de **calcul** utilise `return`. On garde `print` pour la fonction qui **affiche** le rapport final. C'est pour ça que les tests `assert` de l'exercice 6 ne marchent qu'avec `return`.

3. Lire un fichier

Analogie : un fichier, c'est un cahier rangé dans un tiroir. Pour le lire, il faut l'**ouvrir**, le **lire**, puis le **refermer**, sinon le tiroir reste coincé.

Définition 3.1 *(with open)*

```
with open("notes.txt", encoding="utf-8") as fichier:
    contenu = fichier.read()
```

`with` ouvre le fichier **et le referme automatiquement** à la fin du bloc, même en cas d'erreur. `encoding="utf-8"` permet de lire correctement les accents (é, è, à).

MÉTHODE	RENVOIE	USAGE
`f.read()`	tout le texte, en une seule chaîne	petit fichier
`f.readlines()`	une liste de lignes (avec `\n` au bout)	traiter ligne par ligne
`for ligne in f:`	une ligne à la fois	gros fichier : on ne charge pas tout en mémoire

```
notes = []
with open("notes.txt", encoding="utf-8") as f:
    for ligne in f:
        ligne = ligne.strip()          # enlève les espaces et le \n de fin de l
        if ligne:                      # ignore les lignes vides
            notes.append(float(ligne)) # texte → nombre (le piège du Day 2 !)
```

> **Dans le métier**
>
> Toute IA commence par **lire des données** depuis un fichier : CSV, texte, PDF, images. Et tout ce qu'on lit arrive sous forme de **texte** : il faut nettoyer (`strip`) puis convertir (`float`).

4. Écrire un fichier

MODE	EFFET
`"r"`	lire (le mode par défaut)
`"w"`	écrire : **efface** le fichier s'il existe déjà
`"a"`	ajouter à la fin, sans effacer (*append*)

```
with open("rapport.txt", "w", encoding="utf-8") as f:
    f.write("Moyenne : 12.5\n")      # \n = retour à la ligne
    f.write("Admis : 3 / 5\n")
```

Piège

Le mode `"w"` **écrase** sans prévenir. Pour garder l'historique d'un fichier de logs, on utilise `"a"`.

---

5. Les exceptions : `try` / `except`

Analogie : un trapéziste travaille avec un **filet**. S'il tombe, le spectacle continue. `try`, c'est le numéro de trapèze, et `except`, c'est le filet.

Définition 5.1 *(exception)*

Une **exception** est une erreur qui survient pendant l'exécution (`ValueError`, `FileNotFoundError`...). Sans filet, elle arrête le programme. Avec `try` / `except`, on l'**attrape** et on décide quoi faire.

```
try:
    note = float("douze")              # ← ValueError : impossible à convertir
except ValueError:
    print("Note invalide, ignorée")    # le filet : le programme continue
```

Définition 5.2 *(les 4 blocs)*

```
try:
    ...          # le code qui peut échouer
except ValueError as erreur:
    ...          # si ÇA échoue avec une ValueError
else:
    ...          # si TOUT a marché (facultatif)
finally:
    ...          # TOUJOURS exécuté, erreur ou pas (facultatif)
```

```
# Lire des notes « sales » sans jamais planter
valides, rejetees = [], []
for brut in ["12", "15.5", "abc", "", "18"]:
    try:
        valides.append(float(brut))
    except ValueError:
        rejetees.append(brut)
# valides  = [12.0, 15.5, 18.0]
# rejetees = ['abc', '']
```

Piège : le filet trop grand

`except:` tout seul, ou `except Exception:` sans rien afficher, attrape **tout**, y compris tes propres fautes de frappe. Le bug devient invisible. On attrape **l'erreur précise** qu'on attend, et on affiche ou on note ce qui a été ignoré.

Dans ExamPredict

C'est l'exercice 1 de la fiche F3.DAY1 : si un lycéen dépose un PDF corrompu, `try` / `except` permet d'afficher « Fichier ignoré » et de continuer avec les autres, au lieu de tout faire planter.

À dire

« Un programme sans try/except, c'est un trapéziste sans filet : à la première erreur, tout s'arrête. Une vraie IA reçoit des données sales tous les jours, alors on lui met un filet. »

---

6. Lever sa propre erreur : `raise`

Parfois, c'est **toi** qui dois dire « stop, cette donnée est impossible ».

```
def verifier_note(note):
    if not 0 <= note <= 20:
        raise ValueError(f"Note hors limites : {note}")
    return note

verifier_note(25)   # ValueError: Note hors limites : 25
```

En clair : `raise`, c'est l'arbitre qui siffle une faute. Mieux vaut un arrêt clair tout de suite qu'une moyenne fausse découverte trois jours plus tard.

7. Modules et `import`

> **Définition 7.1** *(module)*
>
> Un **module** est un fichier `.py` dont on réutilise les fonctions ailleurs. `import` va chercher ces fonctions.

```
import math                          # module de la bibliothèque standard
math.sqrt(16)                        # 4.0

from collections import Counter      # n'importer qu'un outil précis
from ex06_fonctions import moyenne # importer TON propre fichier
```

MODULE STANDARD	SERT À
`math`	racines, arrondis, constantes
`random`	tirages au hasard (la baseline de la fiche F3)
`collections`	`Counter`, `defaultdict` (fiche F2.DAY1)
`pathlib`	manipuler des chemins de fichiers
`re`	expressions régulières (le bug du « ph »)
`time`	mesurer une durée

8. Le piège de l'import et `__main__`

Dans le bonus `ex07_bonus_bulletin.py`, on importe `ex06_fonctions`. Résultat : le message « ✅ Tous les tests passent » s'affiche **avant** le bulletin. Pourquoi ?

> **Proposition 8.1**
>
> `import` **exécute tout le fichier importé**, de haut en bas : les `def`, mais aussi les `assert` et les `print`.

La solution : `if __name__ == "__main__":`

```
def moyenne(notes):
    return sum(notes) / len(notes)

if __name__ == "__main__":
    # exécuté SEULEMENT si on lance ce fichier directement (py ex06_fonctio
    # PAS exécuté quand un autre fichier fait « import ex06_fonctions »
    assert moyenne([10, 20]) == 15
    print("✅ Tous les tests passent")
```

En clair : `__name__` vaut `"__main__"` quand le fichier est le programme principal. C'est la ligne qu'on trouve tout en bas de `main.py` dans ExamPredict.

---

9. Découper un projet en fichiers

```
analyseur_notes/
├── lecture.py      # lire le fichier, nettoyer, convertir      (sections 3 et
├── stats.py        # moyenne, min, max, médiane                (section 2 : re
├── rapport.py      # afficher et écrire le rapport             (section 4)
└── main.py         # assemble tout : import + __main__         (sections 7 et
```

Règle

Un fichier = une responsabilité. On teste chaque morceau séparément, et on remplace un morceau sans casser les autres. C'est exactement l'architecture d'ExamPredict (fiche F3.DAY1, section 2).

10. Mission : l'analyseur de notes

> **Énoncé (pratique officielle du Day 2)**
>
> Construire un programme qui lit un fichier de notes « sales », calcule des statistiques et ne plante jamais :
>
> 1. `lire_notes(chemin)` lit le fichier ligne par ligne et renvoie `(valides, rejetees)`. Une note est valide si c'est un nombre entre 0 et 20.
> 2. `statistiques(notes)` renvoie un dictionnaire : moyenne, min, max et nombre de notes ≥ 10.
> 3. `ecrire_rapport(stats, rejetees, chemin)` écrit le rapport dans `rapport.txt`.
> 4. Si le fichier n'existe pas, le programme affiche un message clair au lieu de planter (`FileNotFoundError`).
> 5. Des `assert` vérifient les 3 fonctions, dans un bloc `if __name__ == "__main__":`.

11. À savoir par cœur

1	Le paramètre est la case, l'argument est l'ingrédient. Une valeur par défaut, c'est le réglage d'usine.
2	`return` donne la valeur, `print` la montre seulement. Une fonction de calcul utilise `return`.
3	`with open(chemin, encoding="utf-8") as f:` ouvre le fichier et le referme tout seul.
4	Une ligne lue est du texte : `strip()` pour nettoyer, puis `float()` pour convertir.
5	`"w"` efface, `"a"` ajoute, `"r"` lit.
6	`try` / `except ErreurPrécise` est le filet du trapéziste. On n'attrape jamais tout en silence.
7	`raise ValueError("…")` arrête volontairement le programme quand une donnée est impossible.
8	`import` exécute tout le fichier importé : on protège les tests avec `if __name__ == "__main__":`.
9	Un fichier = une responsabilité : lecture, calcul, rapport, main.